Seção 4: Funcionalidades Detalhadas

Esta seção aprofunda-se nas principais funcionalidades do aplicativo Finan, conforme delineado no briefing e enriquecido pelas discussões anteriores. Para cada funcionalidade, serão descritos o objetivo, o cenário de uso, os modelos de dados e campos necessários, os fluxos de tela, estados e interações, os modais, popups e notificações associados, bem como as regras de negócio e validações pertinentes. O intuito é fornecer um guia claro para a implementação de cada recurso, garantindo que a experiência do usuário seja coesa e que os requisitos de negócio sejam atendidos.

4.1. Registro Automático a partir de Notificações Bancárias

O registro automático de transações a partir de notificações bancárias representa uma funcionalidade de grande conveniência, visando simplificar drasticamente o processo de entrada de dados para o usuário e manter suas finanças atualizadas com mínimo esforço.

Objetivo e Cenário de Uso: O objetivo principal desta funcionalidade é capturar automaticamente informações de transações financeiras (compras aprovadas, transferências PIX recebidas ou enviadas, pagamentos de boletos) que são comunicadas ao usuário através de notificações push de seus aplicativos bancários ou de outras instituições financeiras instaladas em seu dispositivo. Em um cenário de uso típico, o usuário realiza uma compra com seu cartão de crédito. Momentos depois, o aplicativo do banco envia uma notificação informando "Compra aprovada no valor de R$XX,XX em Estabelecimento Y". O Finan, com a devida permissão do usuário para ler notificações, detectaria essa informação, processaria os dados relevantes (valor, possível nome do estabelecimento, tipo de transação) e apresentaria uma sugestão de lançamento para o usuário dentro do próprio Finan. O usuário então revisaria a sugestão, faria ajustes se necessário (como confirmar ou alterar a categoria) e confirmaria o lançamento, ou o sistema poderia, opcionalmente e com configuração prévia, realizar o lançamento automaticamente para fontes confiáveis.

Modelos de Dados e Campos Necessários: Para suportar esta funcionalidade, além dos modelos de dados de transação já existentes, seria necessário um modelo para armazenar as notificações capturadas e seu estado de processamento. Poderíamos ter uma coleção ou estrutura temporária para `PendingNotifications` ou `SuggestedTransactions`.

Campos relevantes a serem extraídos ou inferidos da notificação: * `notification_raw_text`: O texto completo da notificação original. *

`source_app_package_name` : Identificador do aplicativo que gerou a notificação (ex: `br.com.itau` , `com.nubank.android`). * `transaction_type_guess` : Inferência do tipo de transação (despesa, receita, transferência). * `amount_detected` : Valor numérico da transação. * `currency_detected` : Moeda (ex: BRL, USD). * `merchant_name_guess` : Nome do estabelecimento ou contraparte, se detectável. * `date_time_detected` : Data e hora da notificação (aproximação da data da transação). * `suggested_category_id` : ID da categoria sugerida pelo sistema com base em regras ou aprendizado. * `status` : (ex: `pending_review` , `confirmed_by_user` , `ignored` , `auto_processed`). * `user_id` : Para associar a notificação/sugestão ao usuário correto.

Fluxos de Telas, Estados e Interações: 1. **Permissão Inicial:** Ao habilitar a funcionalidade pela primeira vez, o aplicativo Finan guiaria o usuário pelo processo de conceder permissão de acesso às notificações do sistema Android ou iOS. Uma tela explicativa detalharia a necessidade da permissão e como ela seria utilizada, reforçando a privacidade e segurança. 2. **Captura em Background:** Um serviço em background (ou listener de notificações) monitoraria as novas notificações. Ao identificar uma notificação de um aplicativo bancário conhecido (lista configurável ou detectada), ele a capturaria. 3. **Parsing e Sugestão:** A notificação capturada seria processada por um parser. Este parser utilizaria expressões regulares, palavras-chave e, potencialmente, um modelo de machine learning simples (treinado com exemplos de notificações) para extrair valor, estabelecimento, e inferir o tipo de transação. Uma sugestão de lançamento seria criada. 4. **Notificação Interna no Finan:** O usuário receberia uma notificação discreta dentro do Finan (ex: um badge no ícone do app ou uma notificação push do próprio Finan) informando que há "Novas sugestões de lançamento para revisar". 5. **Tela de Revisão de Sugestões:** Dentro do Finan, haveria uma seção ou tela específica (ex: "Sugestões Pendentes") onde o usuário veria uma lista das transações sugeridas. Cada item da lista mostraria os dados extraídos e a categoria sugerida. 6. **Interação com a Sugestão:** Ao tocar em uma sugestão, o usuário abriria um modal ou tela similar ao de lançamento manual, mas pré-preenchido com os dados da notificação. O usuário poderia editar qualquer campo (valor, categoria, data, descrição), adicionar anexos e então "Confirmar Lançamento" ou "Ignorar Sugestão". 7. **Confirmação e Lançamento:** Ao confirmar, a transação seria salva como um lançamento normal no Finan, e a sugestão seria marcada como processada.

Modais, Popups, Notificações (Design e Gatilhos): * **Modal de Permissão:** Um modal informativo e claro explicando a necessidade da permissão de acesso às notificações, com um botão para levar o usuário às configurações do sistema. * **Notificação Push do Finan (Opcional):** "Você tem X sugestões de lançamento para revisar no Finan." Gatilho: Após o processamento de novas notificações bancárias. * **Badge no Ícone do App/ Menu:** Indicador visual de sugestões pendentes. * **Modal de Revisão/Edição de Sugestão:** Similar ao modal de lançamento manual, mas com os campos pré-

preenchidos e botões claros para "Confirmar" ou "Ignorar/Descartar". * **Popup de Confirmação (Toast/Snackbar):** "Lançamento confirmado a partir da notificação!" após o usuário aprovar uma sugestão.

Regras de Negócio e Validações: * **Lista de Aplicativos Monitorados:** O sistema deve manter uma lista de package names de aplicativos bancários e financeiros a serem monitorados. Esta lista pode ser atualizada remotamente. * **Prevenção de Duplicidade:** O sistema deve ter mecanismos para evitar sugerir lançamentos duplicados a partir de notificações repetidas ou muito similares. Isso pode envolver a verificação de transações recentes com valor e data próximos. * **Precisão do Parser:** O parser deve ser robusto, mas também reconhecer suas limitações. Se a confiança na extração dos dados for baixa, a sugestão pode indicar isso ao usuário (ex: "Valor detectado, mas verifique a categoria"). * **Privacidade:** O texto completo das notificações só deve ser processado localmente no dispositivo, se possível, ou, se enviado para um backend para parsing avançado (via Cloud Functions, por exemplo), deve ser feito de forma segura e anonimizada, com consentimento explícito do usuário. A política de privacidade deve ser clara sobre como esses dados são tratados. * **Configuração pelo Usuário:** O usuário deve poder habilitar/desabilitar a funcionalidade a qualquer momento e, idealmente, selecionar de quais aplicativos bancários ele deseja que o Finan capture notificações. * **Aprendizado de Categorias (Avançado):** Com o tempo, o sistema poderia aprender as preferências de categorização do usuário para determinados estabelecimentos ou tipos de transação, melhorando a precisão das sugestões de categoria. * **Tratamento de Erros no Parsing:** Se uma notificação não puder ser parseada corretamente, ela deve ser registrada para análise (para melhoria do parser), mas não deve travar o sistema ou apresentar erros ao usuário, a menos que seja uma falha crítica na funcionalidade.

A implementação desta funcionalidade requer um cuidado especial com as permissões do sistema operacional, a diversidade de formatos de notificações bancárias e, acima de tudo, a confiança e privacidade do usuário. Quando bem executada, no entanto, pode ser um diferencial significativo para o aplicativo Finan.

4.2. Planejamento de Contas e Recebíveis

Esta funcionalidade permite aos usuários agendar e acompanhar suas contas a pagar e valores a receber, proporcionando uma visão clara de seus compromissos financeiros futuros e ajudando a evitar atrasos e multas.

Objetivo e Cenário de Uso: O objetivo é oferecer uma ferramenta para que o usuário possa registrar antecipadamente suas despesas fixas ou variáveis com datas de vencimento (como aluguel, contas de água, luz, faturas de cartão de crédito) e também suas receitas esperadas (como salário, aluguéis recebidos, pagamentos de clientes). Em um cenário típico, o usuário cadastra sua conta de internet que vence todo dia 10. O

Finan então o lembraria alguns dias antes e no dia do vencimento. Ao receber a notificação, o usuário poderia marcar a conta como paga (o que poderia, opcionalmente, gerar um lançamento de despesa automático) ou adiar o lembrete. Da mesma forma, ele poderia agendar o recebimento do seu salário para o dia 5 de cada mês, ajudando no planejamento do fluxo de caixa.

Modelos de Dados e Campos Necessários: Será necessária uma nova coleção no Firestore, por exemplo, `scheduled_payments` ou `financial_commitments`, associada a cada usuário.

Campos para cada item agendado: * `user_id`: ID do usuário. * `title`: Descrição breve da conta/recebível (ex: "Aluguel Apartamento", "Salário Mensal"). * `type`: Tipo (`payable` para conta a pagar, `receivable` para valor a receber). * `amount`: Valor estimado ou exato. * `due_date`: Data de vencimento ou data prevista para recebimento. * `category_id`: (Opcional) Categoria associada, para facilitar o lançamento posterior. * `recurrence_rule`: (Opcional) Regra de recorrência (ex: `none`, `monthly_day_10`, `weekly_friday`). Se houver recorrência, o sistema criaria as próximas instâncias automaticamente ou no momento do pagamento da instância atual. * `status`: (ex: `pending`, `paid`, `received`, `overdue`, `cancelled`). * `reminder_preference`: (ex: `1_day_before`, `on_due_date`, `custom`). * `notes`: Anotações adicionais. * `related_transaction_id`: (Opcional) ID da transação no Finan que liquidou este agendamento.

Fluxos de Telas, Estados e Interações: 1. **Acesso à Funcionalidade:** O usuário acessaria a seção "Planejar" através do menu principal. Dentro desta seção, haveria uma sub-seção ou aba para "Contas e Recebíveis". 2. **Visualização dos Agendamentos:** A tela principal desta funcionalidade apresentaria os itens agendados, possivelmente com múltiplas visualizações: * **Lista Cronológica:** Itens ordenados pela data de vencimento/recebimento mais próxima. * **Calendário:** Um calendário mensal onde os dias com agendamentos seriam destacados. Ao selecionar um dia, os agendamentos daquele dia seriam listados abaixo ou em um popup. * Filtros por tipo (a pagar/a receber), status (pendente, pago, vencido). 3. **Adicionar Novo Agendamento:** Um botão "+" ou "Adicionar Novo" levaria a um formulário (modal ou nova tela) para cadastrar uma nova conta ou recebível. 4. **Formulário de Cadastro/Edição:** Campos para título, valor, tipo, data de vencimento/recebimento, categoria (opcional), recorrência (com opções como "Não se repete", "Diariamente", "Semanalmente", "Mensalmente", "Anualmente", e detalhes como dia do mês ou dia da semana), preferências de lembrete e notas. 5. **Interação com Agendamentos:** * **Marcar como Pago/Recebido:** O usuário poderia selecionar um item pendente e marcá-lo como "Pago" ou "Recebido". Esta ação poderia opcionalmente disparar a criação de um lançamento correspondente na lista de transações do Finan (com confirmação do usuário). * **Editar/Excluir:** Opções para

modificar ou remover um agendamento. * **Adiar Lembrete/Vencimento:** Para itens pendentes, opção de adiar o lembrete ou, se aplicável, a data de vencimento (com registro do adiamento). 6. **Tratamento de Recorrências:** Se um item recorrente for marcado como pago, o sistema automaticamente criaria a próxima instância do agendamento com a data futura apropriada, baseada na regra de recorrência.

Modais, Popups, Notificações (Design e Gatilhos): * **Modal de Cadastro/Edição de Agendamento:** Formulário completo com todos os campos necessários, seletores de data, e opções claras para recorrência. * **Notificações Push de Lembrete:** * **Gatilho:** Baseado na `due_date` e `reminder_preference` (ex: 1 dia antes, no dia do vencimento às 9h). * **Conteúdo:** "Lembrete: [Título da Conta] vence hoje/amanhã! Valor: R$XX,XX." ou "Lembrete: Você tem um recebimento de [Título do Recebível] previsto para hoje/amanhã. Valor: R$XX,XX." * **Ações na Notificação (se possível):** "Marcar como Pago", "Adiar Lembrete", "Abrir no Finan". * **Popup de Confirmação:** Ao marcar como pago/recebido, um toast/snackbar confirmaria a ação: "[Título da Conta] marcada como paga!". * **Alerta Visual para Itens Vencidos:** Na lista de agendamentos, itens com `due_date` passada e status `pending` seriam destacados visualmente (ex: cor vermelha, ícone de alerta).

Regras de Negócio e Validações: * **Campos Obrigatórios:** Título, tipo, valor e data de vencimento/recebimento são obrigatórios. * **Validação de Datas:** A data de vencimento/recebimento deve ser uma data válida, geralmente no futuro ou presente para novos agendamentos. * **Lógica de Recorrência:** A geração de instâncias recorrentes deve ser precisa. Se o usuário edita uma série recorrente, ele deve ter a opção de aplicar a mudança apenas à instância atual, a esta e futuras, ou a todas as instâncias. * **Consistência de Status:** Um item não pode ser marcado como `paid` se for do tipo `receivable`, e vice-versa. O status deve refletir o ciclo de vida do agendamento. * **Integração com Lançamentos:** Ao marcar um agendamento como pago/recebido, o sistema deve oferecer a opção de criar automaticamente um lançamento correspondente na seção de transações do Finan. Os dados (valor, data, categoria) seriam pré-preenchidos a partir do agendamento. O usuário confirmaria ou ajustaria antes de salvar o lançamento. * **Notificações:** As notificações devem ser configuráveis pelo usuário (habilitar/desabilitar lembretes gerais, talvez configurar horários preferenciais). * **Performance:** Para usuários com muitos agendamentos recorrentes, a visualização em calendário e a geração de instâncias futuras devem ser otimizadas para não impactar a performance do aplicativo.

Esta funcionalidade de planejamento de contas e recebíveis adiciona uma camada proativa ao gerenciamento financeiro, ajudando os usuários a se manterem organizados e a evitarem surpresas desagradáveis com seus compromissos financeiros.

4.3. Lançamento Manual de Despesas/Receitas

A funcionalidade de lançamento manual é o pilar central de qualquer aplicativo de finanças pessoais, permitindo ao usuário registrar suas transações diárias de forma precisa e categorizada. No Finan, esta funcionalidade será acessível e intuitiva, incentivando o registro consistente.

Objetivo e Cenário de Uso: O objetivo é fornecer uma interface rápida e eficiente para que os usuários registrem todas as suas entradas (receitas) e saídas (despesas) de dinheiro. Em um cenário comum, o usuário almoça em um restaurante e paga R$35,00. Logo após, ele abre o Finan, acessa a função de novo lançamento (idealmente através de um modal deslizante - bottom sheet - na tela inicial, como sugerido no briefing), insere o valor "35,00", seleciona a categoria "Alimentação", confirma a data (que já virá preenchida com o dia atual) e, opcionalmente, adiciona uma nota como "Almoço com cliente" ou anexa uma foto do recibo. Em outro cenário, ao receber seu salário, o usuário registraria o valor como uma receita na categoria "Salário".

Modelos de Dados e Campos Necessários: Esta funcionalidade utilizará o modelo de dados principal `transactions` já definido na Seção 2 (Arquitetura e Conexões), com os seguintes campos chave para cada lançamento manual: * `user_id` : ID do usuário. * `type` : Tipo da transação (`expense` para despesa, `income` para receita). O usuário selecionará isso explicitamente ou através de abas no modal. * `amount` : Valor numérico da transação. Deve ser sempre positivo, e o `type` define se soma ou subtrai do saldo. * `category_id` : ID da categoria selecionada pelo usuário (ex: "Alimentação", "Transporte", "Salário", "Moradia"). * `transaction_date` : Data e hora em que a transação ocorreu. O padrão será a data e hora atuais, mas o usuário poderá alterar. * `description` : (Opcional) Campo de texto para notas, nome do estabelecimento, ou qualquer detalhe adicional. * `location_data` : (Opcional, Avançado) Poderia armazenar dados de geolocalização (latitude, longitude, nome do local) se o usuário permitir e a funcionalidade de "local do gasto" for implementada com captura automática ou busca de locais. * `receipt_image_url` : (Opcional) URL da imagem do recibo armazenada (ex: no Firebase Storage), caso o usuário anexe uma foto. * `creation_method` : (Interno) Indicador de como o lançamento foi criado (ex: `manual` , `from_notification_suggestion` , `from_scheduled_payment`).

Fluxos de Telas, Estados e Interações: 1. **Acesso ao Lançamento:** * **Via FAB na Tela Inicial:** Um botão de ação flutuante (FAB) com um ícone "+" na tela inicial abrirá diretamente o modal de lançamento (bottom sheet). * **Via Tela de Lançamentos:** Um botão "Adicionar Novo" na tela de listagem de lançamentos. * **Contextual:** Em algumas telas (ex: ao confirmar um pagamento agendado), o sistema pode oferecer a criação de um lançamento pré-preenchido. 2. **Modal de Lançamento (Bottom Sheet):** * **Seleção**

de Tipo: Abas no topo do modal para "Despesa" e "Receita". A seleção altera a lógica de categorização e o impacto no saldo. * **Campo de Valor:** Input numérico proeminente para o valor. O teclado numérico deve ser exibido automaticamente. * **Seletor de Categoria:** Um campo que, ao ser tocado, abre um seletor de categorias (pode ser uma nova tela, um modal de lista ou um picker expansível). As categorias exibidas podem ser filtradas pelo tipo de transação (despesa/receita). * **Seletor de Data:** Exibe a data atual por padrão. Ao ser tocado, abre um calendário para o usuário selecionar outra data (e, opcionalmente, hora). * **Campo Descrição/Local:** Campo de texto opcional para o usuário adicionar mais detalhes ou o nome do local do gasto. * **Opção de Anexar Foto:** Um ícone de câmera ou clipe que permite ao usuário tirar uma foto nova ou selecionar uma da galeria do dispositivo para anexar como recibo. * **Botão Salvar:** Botão para confirmar e salvar o lançamento. 3. **Seleção de Categoria:** * Ao tocar no campo de categoria, um modal ou nova tela é apresentado com a lista de categorias disponíveis para o tipo de transação selecionado (despesa ou receita). * A lista deve ser pesquisável e pode incluir as categorias mais usadas no topo. * Opção de "Adicionar Nova Categoria" diretamente deste fluxo, se o usuário não encontrar a desejada (leva para o fluxo de gerenciamento de categorias). 4. **Anexo de Foto:** * Ao selecionar a opção de anexar foto, o aplicativo solicitará permissão para acessar a câmera e/ou galeria (se ainda não concedida). * Após tirar ou selecionar a foto, uma pré-visualização pode ser mostrada, com opção de confirmar ou refazer. * A imagem é então carregada para o Firebase Storage, e a URL é salva no documento da transação. 5. **Confirmação e Feedback:** Após salvar, o modal é fechado, e um feedback visual rápido (toast/snackbar) confirma que o lançamento foi salvo com sucesso (ex: "Despesa de R$35,00 salva!"). A tela inicial ou a lista de lançamentos deve ser atualizada para refletir a nova transação.

Modais, Popups, Notificações (Design e Gatilhos): * **Modal Principal (Bottom Sheet):** Como descrito acima, deve ser projetado para ser rápido e fácil de usar com uma mão. Deve ser responsivo e adaptar-se bem a diferentes tamanhos de tela. * **Modal de Seleção de Categoria:** Lista clara e pesquisável. Ícones podem ser usados para identificar visualmente as categorias. * **Modal de Seleção de Data:** Utilizar o componente de calendário nativo do sistema ou uma biblioteca de calendário bem estabelecida. * **Popup de Confirmação (Toast/Snackbar):** Mensagem breve e auto-dispensável após o salvamento bem-sucedido. * **Alertas de Validação:** Se campos obrigatórios (como valor ou categoria) não forem preenchidos, mensagens de erro inline ou um alerta devem ser exibidos antes de permitir o salvamento. * **Modal de Permissão:** Para acesso à câmera/galeria, se necessário.

Regras de Negócio e Validações: * **Campos Obrigatórios:** Valor, Tipo (Despesa/Receita), Categoria e Data da Transação são obrigatórios. * **Validação de Valor:** O valor deve ser numérico e maior que zero. * **Consistência de Categoria:** A categoria selecionada deve ser compatível com o tipo de transação (uma categoria de "Salário"

não deve ser usada para uma despesa). * **Limite de Tamanho de Anexo:** Definir um limite para o tamanho da imagem do recibo para otimizar o armazenamento e o tempo de upload. * **Formato da Data:** A data deve ser armazenada em um formato padronizado (ex: timestamp ISO 8601) para facilitar consultas e ordenação. * **Feedback Imediato:** O usuário deve ver o impacto do novo lançamento em seu saldo e extratos o mais rápido possível. * **Edição e Exclusão:** Os lançamentos manuais devem poder ser editados ou excluídos posteriormente através da tela de detalhes do lançamento.

O design do fluxo de lançamento manual é crítico para a adoção e uso contínuo do aplicativo. Priorizar a velocidade, a simplicidade e a redução do número de toques necessários para registrar uma transação será fundamental para o sucesso desta funcionalidade.

4.4. Gráficos e Relatórios

Esta funcionalidade é essencial para que os usuários possam visualizar e compreender seus dados financeiros de forma intuitiva, identificar padrões de gastos, acompanhar a evolução de seu patrimônio e tomar decisões financeiras mais informadas. O Finan oferecerá um conjunto de gráficos e relatórios personalizáveis e de fácil interpretação.

Objetivo e Cenário de Uso: O objetivo é transformar dados brutos de transações em insights visuais acionáveis. Em um cenário típico, ao final do mês, o usuário acessa a seção de "Relatórios" para entender como gastou seu dinheiro. Ele visualiza um gráfico de pizza que mostra a distribuição de suas despesas por categoria (ex: 30% em Moradia, 20% em Alimentação, 15% em Transporte, etc.). Ele também pode ver um gráfico de barras comparando suas despesas totais dos últimos três meses ou a evolução de seu saldo líquido. Para uma análise mais detalhada, ele gera um "Relatório Mensal" em PDF que consolida todas as receitas, despesas e o balanço do período selecionado.

Modelos de Dados e Campos Necessários: Esta funcionalidade utilizará principalmente os dados da coleção `transactions` e, potencialmente, dados agregados que podem ser pré-calculados e armazenados para otimizar a performance (ex: em uma coleção `monthly_summaries` por usuário).

Campos relevantes para geração de relatórios (além dos campos de transação): * Para `monthly_summaries` (se implementado): * `user_id` * `month_year` (ex: "2023-10") * `total_income` * `total_expenses` * `balance` * `expenses_by_category`: (Objeto ou array de objetos, ex: `[{category_id: "food", amount: 500}, ...]`).

Fluxos de Telas, Estados e Interações: 1. **Acesso à Funcionalidade:** O usuário acessa a seção "Relatórios" através do menu principal. 2. **Tela Principal de Relatórios:** * **Seleção de Período:** Um seletor proeminente no topo da tela permite ao usuário escolher o

período para o qual deseja visualizar os dados (ex: "Mês Atual", "Mês Anterior", "Últimos 3 Meses", "Ano Atual", ou um intervalo de datas personalizado). * **Dashboard de Relatórios:** A tela apresentará diversos cards ou seções, cada um com um tipo de gráfico ou informação resumida. O usuário poderá interagir com os gráficos (ex: tocar em uma fatia da pizza para ver o valor exato ou as transações daquela categoria). 3. **Tipos de Gráficos e Visualizações:** * **Despesas por Categoria:** Gráfico de Pizza ou Barras mostrando a proporção de gastos em cada categoria para o período selecionado. * **Receitas por Categoria:** Similar ao de despesas, se aplicável (ex: para freelancers com múltiplas fontes de renda). * **Evolução de Despesas e Receitas:** Gráfico de Linhas ou Barras mostrando os totais de despesas e receitas ao longo do tempo (ex: últimos 6 meses, dia a dia no mês atual). * **Comparativo Mensal/Anual:** Gráficos de Barras comparando despesas, receitas ou saldo entre diferentes meses ou anos. * **Fluxo de Caixa:** Gráfico mostrando o saldo diário ou semanal ao longo do período, ilustrando entradas e saídas. * **Visão Geral do Saldo:** Card com o saldo atual, e comparativo com o saldo no início do período selecionado. 4. **Tela de "Relatório Mensal" (ou por período):** * Uma tela dedicada que apresenta um resumo mais formal e detalhado. * **Cabeçalho:** Nome do usuário, período do relatório. * **Seções:** Resumo (Total de Receitas, Total de Despesas, Saldo do Período), Lista detalhada de todas as receitas do período (agrupadas por categoria), Lista detalhada de todas as despesas do período (agrupadas por categoria). 5. **Exportação para PDF:** * Um botão "Exportar para PDF" estará disponível na tela de Relatório Mensal ou na tela principal de Relatórios. * Ao ser acionado, o sistema gerará um arquivo PDF com o relatório formatado. * O PDF incluirá um cabeçalho (logo do Finan, nome do relatório, período) e um rodapé (data de geração, número da página). * O usuário poderá então compartilhar o PDF ou salvá-lo no dispositivo.

Modais, Popups, Notificações (Design e Gatilhos): * **Modal de Seleção de Período Personalizado:** Se o usuário escolher um intervalo de datas personalizado, um modal com dois seletores de data (início e fim) será apresentado. * **Tooltips Interativos nos Gráficos:** Ao passar o dedo ou tocar em elementos do gráfico (barras, fatias de pizza, pontos na linha), um tooltip aparecerá mostrando o valor exato ou informações adicionais. * **Indicador de Carregamento:** Ao gerar gráficos ou relatórios com grande volume de dados, um indicador de carregamento (spinner) será exibido. * **Popup de Confirmação de Exportação:** "Relatório PDF gerado com sucesso! Deseja compartilhar?" com opções de "Compartilhar" ou "Fechar".

Regras de Negócio e Validações: * **Consistência dos Dados:** Os dados exibidos nos gráficos e relatórios devem ser consistentes com os lançamentos registrados pelo usuário. * **Cálculos Precisos:** Todos os totais, médias e percentuais devem ser calculados corretamente. * **Performance na Geração de Gráficos:** Para grandes volumes de transações, a geração de gráficos deve ser otimizada. Isso pode envolver a

agregação de dados no backend (via Cloud Functions) ou o uso eficiente de bibliotecas de gráficos no frontend que suportem virtualização ou processamento assíncrono. * **Tratamento de Dados Vazios ou Insuficientes:** Se não houver dados suficientes para gerar um gráfico significativo (ex: nenhuma despesa no período), uma mensagem amigável deve ser exibida em vez de um gráfico vazio ou um erro (ex: "Você ainda não tem despesas registradas neste período."). * **Layout do PDF:** O layout do PDF exportado deve ser limpo, profissional e legível, com tabelas bem formatadas e gráficos (se incluídos no PDF, o que pode ser uma melhoria futura). * **Personalização (Futuro):** Em fases posteriores, o usuário poderia ter alguma capacidade de personalizar quais gráficos aparecem em seu dashboard de relatórios ou a ordem deles.

Gráficos e relatórios são ferramentas poderosas para engajar os usuários e ajudá-los a extrair valor real de seus dados financeiros. A clareza visual, a precisão dos dados e a facilidade de uso serão cruciais para o sucesso desta funcionalidade no Finan.

4.5. Limites de Gastos e Metas de Economia

Esta funcionalidade capacita os usuários a terem um controle mais proativo sobre suas finanças, permitindo que definam limites para seus gastos em categorias específicas e estabeleçam metas de economia para alcançar objetivos financeiros de curto, médio ou longo prazo.

Objetivo e Cenário de Uso: O objetivo é ajudar os usuários a evitar gastos excessivos e a poupar dinheiro de forma disciplinada. No cenário de limites de gastos, um usuário define que não quer gastar mais de R$500,00 com "Lazer" no mês. Conforme ele registra despesas nessa categoria, o Finan mostra o quanto do limite já foi consumido e o alerta quando ele estiver se aproximando ou ultrapassar o valor definido. Para metas de economia, o usuário pode criar uma meta como "Viagem de Férias - R$3.000,00 até Dezembro". Ele então acompanha o progresso à medida que aloca fundos para essa meta, seja através de transferências específicas ou do saldo economizado de seus orçamentos.

Modelos de Dados e Campos Necessários: Serão necessárias duas novas coleções no Firestore, associadas a cada usuário: 1. `budgets` (para Limites de Gastos): * `user_id` : ID do usuário. * `category_id` : ID da categoria para a qual o limite se aplica. * `amount_limit` : Valor monetário do limite para o período. * `period_type` : Tipo de período (ex: `monthly` , `weekly` , `custom_date_range`). * `start_date` : Data de início do período do orçamento. * `end_date` : Data de fim do período do orçamento. * `current_spending` : (Calculado ou atualizado via trigger) Gasto atual na categoria dentro

do período. * `status`: (ex: `active`, `archived`). * `alert_threshold_percentage`: (Opcional) Percentual do limite para disparar um alerta (ex: 80%, 90%).

1. `savings_goals` (para Metas de Economia):
    - `user_id`: ID do usuário.
    - `goal_name`: Nome da meta (ex: "Viagem de Férias", "Entrada do Carro").
    - `target_amount`: Valor monetário total da meta.
    - `current_amount_saved`: Valor atual economizado para a meta.
    - `target_date`: (Opcional) Data limite para alcançar a meta.
    - `creation_date`: Data em que a meta foi criada.
    - `status`: (ex: `in_progress`, `achieved`, `cancelled`).
    - `icon_name` / `color_code`: (Opcional) Para personalização visual da meta.

Fluxos de Telas, Estados e Interações:

Para Limites de Gastos (Orçamentos): 1. **Acesso e Visualização:** Na seção "Planejar" do menu, o usuário acessa a sub-seção "Orçamentos" ou "Limites de Gastos". A tela exibe uma lista dos limites ativos, mostrando para cada um: nome da categoria, valor do limite, gasto atual (com barra de progresso visual) e valor restante. 2. **Criar/Editar Limite:** Um botão "Adicionar Novo Limite" abre um formulário onde o usuário seleciona a categoria, insere o valor do limite, define o período (mensal por padrão, mas com opção de semanal ou personalizado) e, opcionalmente, um percentual para alerta. 3. **Acompanhamento:** Conforme o usuário registra despesas, o `current_spending` do limite correspondente é atualizado. A barra de progresso na lista de limites reflete essa atualização em tempo real. 4. **Alertas:** Quando o `current_spending` atinge o `alert_threshold_percentage` (ex: 90% do limite) ou ultrapassa o `amount_limit`, uma notificação é enviada ao usuário.

Para Metas de Economia: 1. **Acesso e Visualização:** Também na seção "Planejar", o usuário acessa a sub-seção "Metas de Economia". A tela exibe uma lista das metas, mostrando para cada uma: nome da meta, valor alvo, valor já economizado (com barra de progresso) e, se houver, a data alvo. 2. **Criar/Editar Meta:** Um botão "Adicionar Nova Meta" abre um formulário onde o usuário insere o nome da meta, o valor alvo, e opcionalmente uma data alvo e um ícone/cor para personalização. 3. **Registrar Progresso:** O usuário pode adicionar fundos a uma meta de algumas formas: * **Manualmente:** Selecionando a meta e usando uma opção "Adicionar Dinheiro à Meta", inserindo o valor. * **A partir de Lançamentos (Avançado):** Ao registrar uma receita ou uma "economia" específica, o usuário poderia ter a opção de alocar parte desse valor diretamente a uma meta. 4. **Acompanhamento:** A barra de progresso da meta é atualizada conforme o `current_amount_saved` aumenta.

Modais, Popups, Notificações (Design e Gatilhos): * **Modal de Criação/Edição de Limite:** Campos para categoria, valor, período. Interface clara para definir a recorrência do limite (ex: se é um limite mensal que se renova automaticamente). * **Modal de Criação/Edição de Meta:** Campos para nome, valor alvo, data alvo. * **Modal para Adicionar Dinheiro à Meta:** Input simples para o valor a ser adicionado. * **Notificações Push para Limites:** * **Gatilho:** Gasto atingindo o `alert_threshold_percentage` ou ultrapassando o `amount_limit`. * **Conteúdo:** "Alerta de Orçamento: Você já gastou R$X de R$Y em [Categoria] este mês!" ou "Atenção: Você ultrapassou seu limite de R$Y para [Categoria]!" * **Notificações Push para Metas (Opcional):** * **Gatilho:** Lembretes periódicos para contribuir com a meta, ou ao atingir marcos (ex: 50% da meta alcançada). * **Conteúdo:** "Lembrete: Que tal adicionar um pouco à sua meta '[Nome da Meta]' hoje?" ou "Parabéns! Você alcançou 50% da sua meta '[Nome da Meta]'!" * **Feedback Visual:** Barras de progresso coloridas (ex: verde para bom, amarelo para perto do limite, vermelho para ultrapassado). Animações sutis ao atingir metas ou limites.

Regras de Negócio e Validações:

Para Limites de Gastos: * **Unicidade:** Geralmente, um limite por categoria por período (ex: não se pode ter dois limites mensais para "Alimentação" no mesmo mês). * **Cálculo do Gasto Atual:** Deve somar todas as despesas da categoria especificada que ocorreram dentro do período do limite. * **Renovação Automática:** Limites mensais ou semanais devem ser renovados automaticamente para o próximo período, zerando o `current_spending`. * **Alertas:** Os alertas devem ser oportunos e não excessivos. O usuário deve poder configurar a frequência ou desabilitar alertas específicos.

Para Metas de Economia: * **Valores Positivos:** Valor alvo e valor adicionado devem ser positivos. * **Consistência:** `current_amount_saved` não pode exceder `target_amount` (a menos que se permita "ultrapassar" a meta). * **Edição de Metas:** Se o usuário edita o valor alvo de uma meta em andamento, o progresso percentual deve ser recalculado. * **Arquivamento de Metas:** Metas alcançadas ou canceladas devem ser movidas para uma seção de "arquivadas" em vez de serem excluídas, para manter o histórico.

Estas funcionalidades de limites e metas são cruciais para transformar o Finan de um simples registrador de transações em um verdadeiro parceiro no planejamento financeiro pessoal, incentivando comportamentos financeiros saudáveis.

4.6. Gerenciamento de Categorias

O gerenciamento de categorias é uma funcionalidade fundamental que permite aos usuários organizar suas transações financeiras de maneira personalizada e significativa,

facilitando a análise de gastos e receitas. O Finan oferecerá um sistema flexível para criar, editar, excluir e organizar categorias.

Objetivo e Cenário de Uso: O objetivo é dar ao usuário controle total sobre como suas transações são classificadas. Por padrão, o Finan pode vir com um conjunto de categorias comuns (ex: Alimentação, Transporte, Moradia, Lazer, Salário), mas o usuário deve ser capaz de adaptá-las à sua realidade. Em um cenário, um usuário percebe que gasta muito com "Streaming de Vídeo" e "Streaming de Música", e a categoria genérica "Lazer" não é suficiente. Ele então acessa o gerenciamento de categorias, cria duas novas subcategorias (se a hierarquia for suportada) ou duas novas categorias principais para esses serviços. Em outro caso, ele pode querer renomear uma categoria existente ou excluir uma que não utiliza.

Modelos de Dados e Campos Necessários: Será necessária uma coleção `categories` no Firestore. Esta pode ser uma coleção global com categorias padrão e uma subcoleção por usuário para categorias personalizadas, ou cada usuário ter sua própria lista completa de categorias (padrão + personalizadas).

Campos para cada categoria: * `category_id`: ID único da categoria (gerado automaticamente). * `user_id`: ID do usuário (se as categorias forem específicas do usuário) ou um valor nulo/especial para categorias padrão. * `name`: Nome da categoria (ex: "Alimentação", "Educação"). * `type`: Tipo de transação a que se aplica (`expense`, `income`, ou `both`). * `icon_name`: (Opcional) Nome de um ícone associado à categoria para identificação visual. * `color_code`: (Opcional) Código de cor associado à categoria. * `parent_category_id`: (Opcional, para hierarquia) ID da categoria pai, se for uma subcategoria. * `is_default`: Booleano indicando se é uma categoria padrão do sistema ou criada pelo usuário. * `status`: (ex: `active`, `archived`).

Fluxos de Telas, Estados e Interações: 1. **Acesso à Funcionalidade:** O usuário acessaria a tela de "Gerenciar Categorias" a partir da seção "Mais" no menu principal, ou contextualmente, a partir do seletor de categorias durante o lançamento de uma transação (com uma opção "Adicionar/Gerenciar Categorias"). 2. **Tela de Listagem de Categorias:** * A tela exibiria uma lista de todas as categorias do usuário (padrão e personalizadas), possivelmente separadas por tipo (Despesa/Receita) ou em abas. * Cada item da lista mostraria o nome da categoria, ícone e cor (se definidos). * Um botão "Adicionar Nova Categoria" estaria proeminente. * Opções para editar ou excluir cada categoria personalizada seriam acessíveis (ex: deslizando o item da lista ou através de um menu de contexto). 3. **Modal/Tela de Criação/Edição de Categoria:** * **Campos:** Nome da categoria (obrigatório). Seleção do tipo (Despesa/Receita). Seleção de ícone (a partir de uma biblioteca pré-definida). Seleção de cor (a partir de uma paleta). * **Hierarquia (Opcional):** Se a organização hierárquica for implementada, haveria um seletor para a categoria pai. * **Ações:** Botões "Salvar" e "Cancelar". 4. **Exclusão de**

Categoria: * Ao tentar excluir uma categoria, o sistema verificaria se existem transações associadas a ela. * Se houver transações, o usuário seria alertado e teria opções: * Reatribuir todas as transações dessa categoria para outra categoria existente. * Arquivar a categoria (tornando-a indisponível para novos lançamentos, mas mantendo-a nos lançamentos antigos). * Cancelar a exclusão. * Categorias padrão do sistema geralmente não poderiam ser excluídas, apenas talvez ocultadas ou arquivadas pelo usuário. 5. **Organização (Opcional Avançado):** * **Reordenar:** Permitir que o usuário arraste e solte para reordenar as categorias na lista de seleção. * **Hierarquia:** Implementar um sistema de subcategorias, onde uma categoria principal pode ter várias subcategorias (ex: Lazer > Cinema, Lazer > Shows). Isso permitiria relatórios mais granulares.

Modais, Popups, Notificações (Design e Gatilhos): * **Modal de Criação/Edição de Categoria:** Formulário limpo e intuitivo. A seleção de ícones e cores deve ser visual e fácil. * **Popup de Confirmação de Exclusão:** "Tem certeza que deseja excluir a categoria '[Nome da Categoria]'? X transações estão associadas a ela." com as opções de ação. * **Toast/Snackbar de Feedback:** "Categoria '[Nome]' salva com sucesso!" ou "Categoria excluída."

Regras de Negócio e Validações: * **Nome Único:** O nome da categoria deve ser único para o usuário (ou dentro do mesmo nível hierárquico, se aplicável) e tipo (despesa/receita). * **Campos Obrigatórios:** Nome e tipo da categoria são obrigatórios. * **Impacto da Edição:** Se o nome de uma categoria for editado, todas as transações passadas associadas a ela devem refletir o novo nome nos relatórios e listas. * **Consistência na Exclusão:** Garantir que a lógica de reatribuição ou arquivamento de transações ao excluir uma categoria seja robusta e não cause perda de dados ou inconsistências. * **Categorias Padrão:** As categorias padrão fornecem um bom ponto de partida, mas o usuário deve ter flexibilidade para personalizá-las ou adicionar as suas. * **Limite de Níveis Hierárquicos (se implementado):** Definir um limite razoável para a profundidade das subcategorias (ex: 2 ou 3 níveis) para evitar complexidade excessiva.

Um sistema de gerenciamento de categorias bem implementado é vital para a personalização e eficácia do Finan, permitindo que os usuários moldem o aplicativo às suas necessidades financeiras específicas.

4.7. Funcionalidades Extras

Esta subseção agrupa um conjunto de funcionalidades adicionais que podem enriquecer a experiência do usuário do Finan, oferecendo conveniência, inovação e maior utilidade. A implementação destas funcionalidades pode ser priorizada para fases mais avançadas do roadmap (Fase 3), após a consolidação dos recursos principais.

4.7.1. Detecção de Compras via Acelerômetro (Movimento Brusco)

Objetivo e Cenário de Uso: O objetivo é oferecer uma forma alternativa e rápida de iniciar um lançamento de despesa, baseada em um gesto físico que o usuário pode associar a uma compra rápida. A ideia, conforme o RF14 original, é que um movimento brusco específico detectado pelo acelerômetro do dispositivo (como um "tapinha" no celular após uma compra) poderia acionar a abertura do modal de novo lançamento. Este recurso visa capturar aquelas pequenas despesas do dia a dia que podem ser esquecidas se não registradas imediatamente.

Modelos de Dados e Campos Necessários: Não requer novos modelos de dados significativos, pois apenas iniciaria o fluxo de lançamento manual. Poderia haver configurações de usuário para habilitar/desabilitar esta funcionalidade e ajustar a sensibilidade do gesto.

Fluxos de Telas, Estados e Interações: 1. **Habilitação e Configuração:** O usuário habilitaria a funcionalidade nas configurações do aplicativo. Poderia haver uma tela para "calibrar" ou aprender o gesto desejado, ou o aplicativo usaria um padrão de movimento pré-definido. 2. **Monitoramento em Background (Controlado):** Com a funcionalidade ativa e o aplicativo em um estado apropriado (talvez não enquanto o usuário está em uma chamada ou usando ativamente outro app), o Finan monitoraria os dados do acelerômetro para o padrão de movimento configurado. 3. **Detecção do Gesto:** Ao detectar o gesto, o Finan traria para o primeiro plano (se não estiver) e abriria diretamente o modal de lançamento de despesa, pronto para o usuário inserir os detalhes.

Modais, Popups, Notificações: * **Modal de Lançamento:** O mesmo modal de lançamento manual seria utilizado. * **Feedback Tátil/Visual (Opcional):** Uma vibração sutil ou um feedback visual rápido poderia confirmar que o gesto foi reconhecido antes de abrir o modal.

Regras de Negócio e Validações: * **Prevenção de Falsos Positivos:** A maior dificuldade é garantir que o gesto seja específico o suficiente para não ser acionado acidentalmente por movimentos normais do dia a dia. A sensibilidade e o padrão do gesto precisam ser cuidadosamente ajustados. * **Consumo de Bateria:** O monitoramento contínuo do acelerômetro pode consumir bateria. A implementação deve ser otimizada, talvez ativando o monitoramento apenas em contextos específicos ou por períodos limitados após o aplicativo ser aberto. * **Configuração do Usuário:** O usuário deve ter controle total sobre esta funcionalidade, incluindo a capacidade de desabilitá-la facilmente.

4.7.2. Upload de Foto de Recibo e OCR para Preenchimento Automático

Objetivo e Cenário de Uso: O objetivo é simplificar ainda mais o lançamento de despesas, permitindo que o usuário tire uma foto de um recibo e o sistema, utilizando tecnologia de Reconhecimento Óptico de Caracteres (OCR), extraia automaticamente informações como valor, nome do estabelecimento e data, pré-preenchendo o formulário de lançamento.

Modelos de Dados e Campos Necessários: Utiliza o campo `receipt_image_url` no modelo de `transactions`. O serviço de OCR pode retornar um JSON com os campos extraídos.

Fluxos de Telas, Estados e Interações: 1. **Início do Fluxo:** No modal de lançamento manual, ao invés de apenas anexar a foto, haveria uma opção "Escanear Recibo com OCR". 2. **Captura da Imagem:** O usuário tira uma foto do recibo. Dicas na tela podem ajudar a obter uma imagem de boa qualidade para o OCR (boa iluminação, recibo plano). 3. **Processamento OCR:** A imagem é enviada para um serviço de OCR (pode ser uma Cloud Function que integra com uma API de OCR como Google Cloud Vision AI, ou uma biblioteca de OCR no dispositivo, embora esta última possa ser menos precisa e mais pesada). 4. **Pré-preenchimento:** Os dados extraídos pelo OCR (valor, data, nome do estabelecimento) são usados para pré-preencher os campos no modal de lançamento. 5. **Revisão e Confirmação:** O usuário revisa os campos pré-preenchidos, corrige se necessário (pois o OCR não é 100% preciso), seleciona a categoria e salva o lançamento.

Modais, Popups, Notificações: * **Interface da Câmera:** Com guias ou sobreposições para ajudar no enquadramento do recibo. * **Indicador de Processamento:** Enquanto o OCR está trabalhando. * **Feedback sobre Precisão (Opcional):** Se o OCR tiver baixa confiança em algum campo, ele pode ser destacado para revisão.

Regras de Negócio e Validações: * **Qualidade da Imagem:** A precisão do OCR depende muito da qualidade da imagem. * **Serviço de OCR:** A escolha do serviço de OCR impactará o custo e a precisão. APIs baseadas em nuvem tendem a ser mais precisas. * **Privacidade:** Se as imagens dos recibos forem enviadas para um serviço de nuvem, a política de privacidade deve cobrir isso. * **Fallback:** Se o OCR falhar ou extrair dados incorretos, o usuário deve poder preencher manualmente sem problemas.

4.7.3. Importação/Exportação de Dados (CSV/JSON) para Sincronização entre Dispositivos

Objetivo e Cenário de Uso: O objetivo é permitir que os usuários façam backup de seus dados financeiros ou os transfiram para outras ferramentas ou dispositivos. A sincronização entre dispositivos já é parcialmente coberta pelo uso do Firestore (se o

usuário logar com a mesma conta em múltiplos dispositivos, os dados estarão sincronizados). Esta funcionalidade foca mais na exportação para formatos abertos e na importação a partir deles.

Modelos de Dados e Campos Necessários: Os dados a serem importados/exportados são principalmente as transações, categorias, orçamentos e metas.

Fluxos de Telas, Estados e Interações: 1. **Acesso:** Na seção "Mais" ou "Configurações", haveria opções para "Importar Dados" e "Exportar Dados". 2. **Exportação:** * O usuário seleciona o que deseja exportar (ex: transações, todas as categorias) e o período (para transações). * Seleciona o formato (CSV ou JSON). * O aplicativo gera o arquivo e oferece opções para salvá-lo no dispositivo ou compartilhá-lo (e-mail, nuvem). 3. **Importação:** * O usuário seleciona o arquivo CSV ou JSON a ser importado (usando um seletor de arquivos do sistema). * O aplicativo analisa o arquivo, mostra uma prévia dos dados a serem importados e pede confirmação. * Pode haver um mapeamento de campos se o formato do arquivo importado não for exatamente o esperado. * O sistema processa a importação, tratando duplicatas ou conflitos conforme regras definidas (ex: ignorar duplicatas, sobrescrever, etc.).

Modais, Popups, Notificações: * **Modal de Opções de Exportação/Importação.** * **Indicador de Progresso:** Para operações de importação/exportação demoradas. * **Relatório de Importação:** Após a importação, um resumo do que foi importado, o que falhou e porquê.

Regras de Negócio e Validações: * **Formato dos Arquivos:** Definir claramente o formato esperado para CSV (colunas, delimitador) e JSON (estrutura). * **Tratamento de Erros na Importação:** Lidar com arquivos malformados, dados inválidos, etc. * **Prevenção de Duplicatas:** Mecanismos para evitar a importação de transações ou outros dados que já existem. * **Segurança:** Alertar o usuário sobre os riscos de importar dados de fontes não confiáveis.

Estas funcionalidades extras, embora complexas, podem adicionar um valor considerável ao Finan, tornando-o uma ferramenta financeira ainda mais completa e adaptada às necessidades variadas dos usuários.

Seção 5: UI/UX e Acessibilidade

Esta seção detalha as diretrizes e recomendações para o design da Interface do Usuário (UI), a Experiência do Usuário (UX) e as práticas de Acessibilidade a serem adotadas no desenvolvimento do aplicativo Finan. O objetivo é criar uma interface que não seja apenas visualmente agradável, mas também intuitiva, eficiente, prazerosa de usar e acessível a todos os usuários, independentemente de suas habilidades ou limitações.

Padrões Visuais (Guia de Estilo)

A consistência visual é fundamental para criar uma identidade de marca forte e uma experiência de usuário coesa. Um guia de estilo visual conciso será desenvolvido e seguido.

- **Paleta de Cores:**
  - **Cor Primária:** Uma cor principal será escolhida para representar a marca Finan (ex: um tom de verde associado a finanças e crescimento, ou um azul que transmita confiança e segurança). Esta cor será usada em elementos de destaque, como cabeçalhos, botões de ação principais e ícones importantes.
  - **Cores Secundárias:** Duas a três cores secundárias que complementem a cor primária serão definidas. Elas podem ser usadas para fundos de cards, elementos de navegação secundários ou para diferenciar seções.
  - **Cores de Acento/Feedback:** Cores específicas para feedback ao usuário:
    - Sucesso (ex: verde vibrante para confirmações).
    - Alerta/Atenção (ex: amarelo ou laranja para avisos, proximidade de limites).
    - Erro/Perigo (ex: vermelho para erros de validação, orçamentos estourados).
    - Cores Neutras: Tons de cinza (claro, médio, escuro) para textos, bordas, fundos e elementos de interface menos proeminentes. Um branco ou off-white para fundos principais, garantindo boa legibilidade.
  - **Contraste:** Todas as combinações de cores de texto e fundo deverão atender aos mínimos de contraste recomendados pelas diretrizes de acessibilidade (WCAG AA, idealmente AAA onde possível).
- **Tipografia:**
  - **Fonte Principal:** Será escolhida uma fonte sans-serif limpa e legível, adequada para interfaces digitais e disponível em diferentes pesos (ex: Roboto, Open Sans, Montserrat). A mesma família de fontes será usada em todo o aplicativo para consistência.
  - **Hierarquia Tipográfica:** Serão definidos tamanhos e pesos de fonte para diferentes elementos textuais (títulos de tela, subtítulos, corpo de texto, legendas, botões) para criar uma hierarquia visual clara e facilitar a escaneabilidade do conteúdo.
  - **Espaçamento de Linha e Letra:** O espaçamento entre linhas (leading) e entre letras (tracking) será ajustado para otimizar a legibilidade em diferentes tamanhos de tela.

- **Espaçamentos e Grid:**
    - Será adotado um sistema de espaçamento consistente (ex: baseado em múltiplos de 4 ou 8 pixels/dp) para margens, paddings e o espaço entre elementos da UI. Isso garante um layout harmonioso e previsível.
    - Um grid layout pode ser definido para alinhar os elementos nas telas, especialmente em telas com mais densidade de informação.
- **Iconografia:**
    - Será utilizado um conjunto de ícones coeso e estilisticamente similar. Bibliotecas como Material Community Icons, Font Awesome (via `react-native-vector-icons`) ou um conjunto de ícones customizados podem ser usados.
    - Os ícones devem ser intuitivos e facilmente reconhecíveis, reforçando a ação ou o conteúdo que representam.

Guia para Animações Sutis e Microinterações

Animações e microinterações, quando usadas com moderação e propósito, podem melhorar significativamente a experiência do usuário, fornecendo feedback, guiando o foco e adicionando um toque de refinamento à interface.

- **Transições de Tela:** As transições entre telas devem ser suaves e consistentes com a plataforma (ex: slide horizontal para navegação em stack, fade para modais). Evitar animações excessivamente longas ou complexas que possam distrair ou causar lentidão.
- **Feedback de Toque:** Elementos interativos (botões, itens de lista) devem fornecer feedback visual imediato ao serem tocados (ex: mudança sutil de cor de fundo, efeito de ondulação - ripple effect).
- **Indicadores de Carregamento:** Para operações assíncronas que podem levar algum tempo (carregar dados, processar uma imagem), indicadores de carregamento claros (spinners, barras de progresso) devem ser exibidos para informar ao usuário que o sistema está trabalhando.
- **Mudanças de Estado:** Animações sutis podem ser usadas para indicar mudanças de estado, como um item sendo adicionado a uma lista, um contador sendo atualizado, ou um alerta aparecendo.
- **Validação de Formulários:** Feedback animado para erros de validação (ex: um leve tremor no campo de input incorreto) pode ser mais eficaz do que apenas texto de erro estático.
- **Foco e Atenção:** Animações podem ser usadas para direcionar a atenção do usuário para elementos importantes, como um novo item em uma lista ou uma notificação.

- **Princípios:** As animações devem ser:
    - **Propositais:** Devem servir a um propósito claro (feedback, orientação, deleite sutil) e não serem meramente decorativas.
    - **Rápidas e Responsivas:** Não devem atrasar a interação do usuário.
    - **Consistentes:** Seguir um padrão em todo o aplicativo.
    - **Opcionais/Reduzíveis:** Respeitar as configurações de acessibilidade do sistema operacional para reduzir movimento, se o usuário assim preferir.

Recomendações de Acessibilidade (WCAG)

Garantir que o Finan seja acessível a todos, incluindo pessoas com deficiências (visuais, auditivas, motoras, cognitivas), é uma prioridade. As seguintes recomendações, baseadas nas Web Content Accessibility Guidelines (WCAG), serão seguidas:

1. **Perceptível:**
    - **Alternativas Textuais:** Todas as imagens não textuais (ícones sem texto, imagens de recibos) devem ter alternativas textuais (`accessibilityLabel` no React Native) que descrevam seu propósito.
    - **Contraste de Cores:** Como mencionado, o contraste entre texto e fundo deve ser suficiente (mínimo 4.5:1 para texto normal, 3:1 para texto grande).
    - **Tamanho de Texto Ajustável:** O aplicativo deve respeitar as configurações de tamanho de fonte do sistema operacional, ou oferecer uma opção interna para ajustar o tamanho do texto.
    - **Não Confiar Apenas na Cor:** Informações importantes não devem ser transmitidas usando apenas cor (ex: um status de erro deve ter um ícone ou texto além da cor vermelha).
2. **Operável:**
    - **Navegação via Teclado (para plataformas que suportam):** Embora primariamente mobile, se houver uso em emuladores ou dispositivos com teclado, todos os elementos interativos devem ser alcançáveis e operáveis via teclado.
    - **Foco Visível:** Elementos interativos devem ter um indicador de foco claro quando selecionados.
    - **Áreas de Toque Suficientes:** Botões e outros controles interativos devem ter uma área de toque grande o suficiente (mínimo 44x44 dp recomendado) para serem facilmente acionados.
    - **Gestos Simples:** Evitar depender de gestos complexos que possam ser difíceis para alguns usuários. Fornecer alternativas se gestos forem usados.

- **Controle de Animações:** Permitir que os usuários pausem, parem ou ocultem animações que piscam ou se movem, ou respeitar as configurações do sistema para "reduzir movimento".

3. **Compreensível:**
    - **Linguagem Clara e Simples:** Usar linguagem fácil de entender, evitando jargões financeiros complexos sem explicação.
    - **Navegação Consistente:** A navegação e a disposição dos elementos devem ser consistentes em todo o aplicativo.
    - **Labels e Instruções Claras:** Todos os campos de formulário devem ter labels claras. Instruções para tarefas complexas devem ser fornecidas.
    - **Prevenção e Correção de Erros:** Mensagens de erro devem ser claras, específicas e sugerir como corrigir o problema. Validação de entrada deve ajudar a prevenir erros.

4. **Robusto:**
    - **Compatibilidade com Tecnologias Assistivas:** O aplicativo deve ser desenvolvido de forma a ser compatível com leitores de tela (ex: VoiceOver no iOS, TalkBack no Android) e outras tecnologias assistivas. Isso envolve o uso correto de propriedades de acessibilidade (`accessibilityLabel` , `accessibilityHint` , `accessibilityRole` , `accessibilityState` no React Native).
    - **Testes com Usuários com Deficiência:** Se possível, realizar testes de usabilidade com pessoas com diferentes tipos de deficiência para identificar e corrigir barreiras de acessibilidade.

Ao integrar estas diretrizes de UI/UX e acessibilidade desde o início do processo de design e desenvolvimento, o Finan poderá oferecer uma experiência de alta qualidade para uma base de usuários mais ampla e diversificada.

Seção 6: Critérios de Qualidade e Testes

Esta seção estabelece os critérios de qualidade que o aplicativo Finan deve atender e descreve a estratégia de testes a ser adotada durante o ciclo de desenvolvimento. O objetivo é garantir que o aplicativo seja robusto, confiável, seguro, performático e que atenda às expectativas dos usuários e aos requisitos de negócio definidos.

Estratégia de Testes

Uma abordagem de testes multifacetada será implementada, cobrindo diferentes níveis e tipos de testes para assegurar a qualidade do software de forma abrangente.

- **Testes Unitários:**
  - **Objetivo:** Verificar a menor unidade de código testável (funções, métodos, componentes individuais) de forma isolada para garantir que ela funcione conforme o esperado.
  - **Escopo:** No frontend (React Native), serão testados componentes de UI (verificando renderização e comportamento básico), funções utilitárias, hooks customizados e lógica de gerenciamento de estado (reducers, actions, selectors se Redux for usado). No backend (Cloud Functions), cada função será testada individualmente para validar sua lógica de negócios, manipulação de dados e tratamento de erros.
  - **Ferramentas:** Para React Native, **Jest** em conjunto com **React Native Testing Library** será a principal combinação para testes unitários e de componentes. Para Cloud Functions (Node.js/TypeScript), **Jest** ou **Mocha/Chai** podem ser utilizados.
  - **Cobertura:** O objetivo será alcançar uma cobertura de testes unitários significativa para as partes críticas e complexas do código. A cobertura de código será monitorada como uma métrica, mas o foco principal será na qualidade e relevância dos testes.
- **Testes de Integração:**
  - **Objetivo:** Verificar a interação entre diferentes componentes ou módulos do sistema para garantir que funcionem juntos corretamente. Isso inclui a integração entre componentes do frontend, a comunicação do frontend com o backend (Firebase SDK, Cloud Functions) e a interação entre diferentes Cloud Functions.
  - **Escopo:**
    - **Frontend:** Testar a navegação entre telas, o fluxo de dados entre componentes pais e filhos, e a integração com serviços locais (como o gerenciador de estado ou o acesso a APIs nativas via Expo).
    - **Backend:** Testar a interação de Cloud Functions com o Firestore (leitura, escrita, triggers) e com o Firebase Authentication.
    - **Frontend-Backend:** Testar os fluxos completos que envolvem chamadas do aplicativo para o Firebase, como o processo de login, registro de uma nova transação, ou a busca de dados para relatórios.

Isso pode envolver o uso de emuladores do Firebase para criar um ambiente de teste controlado.
    - **Ferramentas:** React Native Testing Library pode ser usada para testes de integração de componentes no frontend. Para a integração frontend-backend, testes end-to-end (descritos abaixo) podem cobrir parte disso, ou podem ser criados testes específicos que mockam apenas as camadas externas (ex: rede) mas testam a integração real com os emuladores do Firebase.

- **Testes End-to-End (E2E) / Testes de UI:**
    - **Objetivo:** Simular cenários de uso reais do usuário, testando o aplicativo como um todo, desde a interface do usuário até o backend, para validar os fluxos de funcionalidades completas.
    - **Escopo:** Cobrir os principais fluxos de usuário, como: criar uma conta, fazer login, registrar uma despesa, registrar uma receita, visualizar um relatório, definir um orçamento, adicionar uma meta de economia, etc.
    - **Ferramentas:** Para React Native, frameworks como **Detox** ou **Appium** são comumente usados para testes E2E em dispositivos reais ou emuladores/simuladores.
    - **Automação:** O objetivo é automatizar o máximo possível dos testes E2E para garantir que regressões não sejam introduzidas em funcionalidades existentes.

- **Testes de Usabilidade e Experiência do Usuário (UX):**
    - **Objetivo:** Avaliar a facilidade de uso, a intuitividade da interface e a satisfação geral do usuário ao interagir com o aplicativo.
    - **Metodologia:** Realizar sessões de teste com usuários reais (ou representativos do público-alvo), onde eles executam tarefas específicas no aplicativo. Observar suas interações, dificuldades e coletar feedback qualitativo. Isso pode ser feito em diferentes estágios do desenvolvimento, desde protótipos até versões beta.

- **Testes de Performance:**
    - **Objetivo:** Garantir que o aplicativo seja responsivo, rápido e não consuma recursos excessivos do dispositivo (CPU, memória, bateria, dados de rede).
    - **Escopo:** Medir tempos de carregamento de tela, responsividade da UI a interações, performance de consultas ao banco de dados (especialmente para relatórios e listas longas), e consumo de bateria para funcionalidades em background (como o monitoramento de notificações).

    - **Ferramentas:** Ferramentas de profiling do React Native (Flipper, React DevTools Profiler), Firebase Performance Monitoring, e ferramentas de profiling nativas do Android Studio e Xcode.

- **Testes de Segurança:**
    - **Objetivo:** Identificar e mitigar vulnerabilidades de segurança no aplicativo e no backend.
    - **Escopo:** Revisão das regras de segurança do Firestore e Firebase Authentication, verificação de como dados sensíveis são armazenados e transmitidos, testes de penetração básicos (se aplicável e dentro do orçamento), e análise de dependências para vulnerabilidades conhecidas.

- **Testes de Acessibilidade:**
    - **Objetivo:** Verificar se o aplicativo atende às diretrizes de acessibilidade (WCAG) e é utilizável por pessoas com diferentes tipos de deficiência.
    - **Metodologia:** Utilizar ferramentas de verificação automática de acessibilidade, testes manuais com leitores de tela (VoiceOver, TalkBack) e, idealmente, testes com usuários com deficiência.

Métricas de Sucesso (KPIs) por Funcionalidade

Definir Indicadores Chave de Performance (KPIs) ajudará a medir o sucesso de cada funcionalidade e do aplicativo como um todo, orientando futuras melhorias. Os KPIs devem ser SMART (Específicos, Mensuráveis, Alcançáveis, Relevantes, Temporais).

1. **Aquisição e Ativação de Usuários:**
    - Número de downloads do aplicativo.
    - Taxa de conclusão do processo de cadastro (usuários que iniciam vs. completam o cadastro).
    - Número de usuários ativos diários (DAU) e mensais (MAU).
    - Tempo médio para o usuário realizar o primeiro lançamento de transação após o cadastro.
2. **Engajamento com Funcionalidades Principais:**
    - **Lançamento de Transações:** Número médio de transações lançadas por usuário por semana/mês. Percentual de usuários ativos que lançam pelo menos X transações por semana.
    - **Registro Automático via Notificações:** Taxa de adoção da funcionalidade (usuários que habilitam). Número de transações sugeridas vs. confirmadas. Tempo economizado pelo usuário (estimado).

- **Planejamento (Contas/Recebíveis):** Número de contas/recebíveis agendados por usuário. Taxa de lembretes que resultam em ação (marcar como pago/recebido).
- **Gráficos e Relatórios:** Frequência de acesso à seção de relatórios. Tempo médio gasto na seção. Taxa de exportação de relatórios PDF.
- **Limites de Gastos:** Percentual de usuários ativos que definem pelo menos um limite. Taxa de sucesso em permanecer dentro dos limites definidos.
- **Metas de Economia:** Número de metas criadas por usuário. Taxa de conclusão de metas. Valor médio economizado por meta.
- **Gerenciamento de Categorias:** Número de categorias personalizadas criadas por usuário.

3. **Retenção de Usuários:**
    - Taxa de retenção de usuários (D1, D7, D30 - percentual de usuários que retornam após 1, 7, 30 dias).
    - Taxa de churn (percentual de usuários que deixam de usar o aplicativo).
4. **Qualidade e Performance do Aplicativo:**
    - Taxa de crash do aplicativo (número de crashes por sessão ou por usuário).
    - Tempo médio de carregamento das telas principais.
    - Avaliação média do aplicativo nas lojas (App Store, Google Play).
    - Número de tickets de suporte relacionados a bugs ou problemas de usabilidade.
5. **Satisfação do Usuário:**
    - Net Promoter Score (NPS) coletado através de pesquisas no aplicativo.
    - Feedback qualitativo de reviews nas lojas e canais de suporte.

Monitoramento e Análise: Estas métricas serão monitoradas utilizando ferramentas como Firebase Analytics, Google Analytics for Firebase, e possivelmente plataformas de análise de produto mais especializadas. A análise regular desses KPIs permitirá identificar pontos fortes e fracos do aplicativo, entender o comportamento do usuário e tomar decisões baseadas em dados para o desenvolvimento de novas funcionalidades e aprimoramento das existentes.

A combinação de uma estratégia de testes robusta com o acompanhamento de métricas de sucesso claras será fundamental para garantir que o Finan não apenas funcione corretamente, mas também entregue valor real e uma experiência de alta qualidade aos seus usuários.